

Architectural Justification

TOPIC: Database Query Performance Profiler (44)

Name: Samiksha B N

SRN: PES1UG24CS413

Section: 5G

Architecture Selection:

We chose Microservices Architecture for the Database Query Performance Profiler system.

Reason 1: Independent Scaling for High-Throughput Log Ingestion

NFR-001 requires the system to ingest up to 5,000 slow-query log records per minute without dropping entries or slowing the host database. The Log Ingestion Component has a fundamentally different load profile from the Dashboard/UI or Alert & Digest components, it needs to scale horizontally during traffic spikes. A Microservices architecture lets the ingestion service scale independently (e.g., run multiple instances) without over-provisioning the rest of the system, something a single-deployment Layered or Client-Server architecture cannot do without scaling the entire application.

Reason 2: Fault Isolation Between Independent Concerns

The Query Analyzer, Alert & Digest Service, and Log Ingestion each run as independently deployable services. If the weekly digest generation job fails or is slow, it must not block real-time critical alerts (FR-005) or delay ingestion of new logs, these have very different urgency profiles. Microservices' process isolation ensures a fault or deployment in one service does not cascade into the others, which is critical since real-time alerting has a much tighter latency requirement than weekly digest reporting.

Security Advantage

Isolating the Log Ingestion Component as its own service means only that service handles raw slow-query log text. The Dashboard & Alerting UI Component never has direct access to raw logs, it only consumes processed recommendations and alerts through the Query Analyzer's interfaces. This reduces the attack surface: a compromise of the user-facing dashboard does not expose raw database log content, since that data boundary is enforced by the service split itself.

Performance Benefit

Because ingestion, analysis, alerting, and the dashboard are separate services, the Log Ingestion Component can be scaled and optimized purely for high-volume write throughput (satisfying the 5,000 records/minute requirement) without that load ever competing for resources with the Dashboard & Alerting UI Component that the Database Administrator interacts with directly. This keeps the DBA-facing dashboard responsive even during peak log ingestion periods, which a single monolithic deployment sharing one resource pool would not guarantee.